

Table 1: Pass-sum execution time (s, median per iteration; sum of pass medians, not full executable wall time) and speedup split by pass type. End-to-end includes the build pass when instrumented. When present, the OctTree row includes ColorOctree passes; a separate ColorOctree row reports only those passes. ADT fields = non-recursive fields annotated with `@BENCH adt_fields`; SoA bufs = 1+ non-recursive field slots across all constructors (recursive children are stored in the tag buffer). Speedup $>1\times$ means SoA is faster; **bold** marks $>1.1\times$.

Program	ADT	SoA	End-to-end			Fold passes			Map passes		
	fields	bufs	AoS (s)	SoA (s)	Speedup	AoS (s)	SoA (s)	Speedup	AoS (s)	SoA (s)	Speedup
OctTree	16	9	3.825	8.083	0.47 $\times$	0.3794	0.3275	1.16 $\times$	0.3038	0.3483	0.87 $\times$
Compiler	9	8	0.4941	0.4275	1.16 $\times$	0.1284	0.0391	3.29 $\times$	0.1595	0.1802	0.89 $\times$
DBQuery	15	13	0.4769	0.4882	0.98 $\times$	0.1210	0.0872	1.39 $\times$	0.1380	0.1662	0.83 $\times$
DecisionTree	7	6	4.270	3.987	1.07 $\times$	3.850	3.547	1.09 $\times$	—	—	—
DomTree	15	14	0.4330	0.5235	0.83 $\times$	0.1389	0.0636	2.18 $\times$	0.0585	0.0786	0.74 $\times$
KDTree	17	16	0.4344	0.3107	1.40 $\times$	0.1793	0.1394	1.29 $\times$	—	—	—
LinearListReduction	11	11	0.3461	0.3371	1.03 $\times$	0.0316	0.0050	6.34 $\times$	—	—	—
List	2	2	0.9402	1.108	0.85 $\times$	0.3272	0.3070	1.07 $\times$	0.2516	0.3393	0.74 $\times$
MonoTree	3	2	0.2451	0.1219	2.01 $\times$	0.0175	0.0223	0.78 $\times$	0.0457	0.0422	1.08 $\times$
ObjectGraph	5	4	0.3092	0.4636	0.67 $\times$	0.0913	0.0825	1.11 $\times$	0.1428	0.1620	0.88 $\times$
PiecewiseFunctions	8	7	0.6211	0.6073	1.02 $\times$	0.1357	0.0911	1.49 $\times$	0.2364	0.2566	0.92 $\times$
TernaryTree	5	3	0.3270	0.3283	1.00 $\times$	0.0304	0.0289	1.05 $\times$	0.0840	0.0775	1.08 $\times$
Trie	10	9	0.4213	0.4108	1.03 $\times$	0.0644	0.0319	2.02 $\times$	0.1496	0.1678	0.89 $\times$
ColorOctree	25	18	0.4706	0.7767	0.61 $\times$	0.0918	0.0766	1.20 $\times$	—	—	—

Table 2: Baseline Gibbon comparison (times only). Shown variants: AoS-mut, AoS-imm, SoA-mut. Times are full executable end-to-end wall time per run (s), not pass-sum time. **Bold** marks the fastest time across shown variants.

Program	Am	Ai	Sm
Compiler	3.544	—	2.996
DBQuery	3.321	—	3.200
DecisionTree	43.42	—	39.46
DomTree	2.702	—	2.183
KDTree	2.232	—	1.867
LinearListReduction	0.6698	—	0.5109
List	7.022	—	7.950
MonoTree	1.023	—	0.9381
ObjectGraph	2.918	—	3.294
Oct:sumMass	0.7548	—	1.170
Oct:sumEnergy	0.8106	—	1.216
Oct:countActive	0.7891	—	1.186
Oct:countParticles	0.9111	—	1.147
Oct:barnesHutPotential	0.8620	—	1.409
Oct:fmmPotential	1.445	—	1.906
Oct:scaleEnergy	2.154	—	3.066
Oct:clearFlags	2.190	—	2.936
PiecewiseFunctions	4.561	—	4.558
TernaryTree	1.397	—	1.471
Trie	2.576	—	2.700
ColorOctree	1.314	—	1.563

Table 3: Baseline Gibbon comparison (speedups). Shown: AoS-mut/SoA-mut, AoS-imm/AoS-mut, AoS-imm/SoA-mut. Speedups are computed from full executable end-to-end wall time per run. $>1\times$ means the denominator is faster.

Program	Am/Sm	Ai/Am	Ai/Sm
Compiler	1.18 $\times$	—	—
DBQuery	1.04 $\times$	—	—
DecisionTree	1.10 $\times$	—	—
DomTree	1.24 $\times$	—	—
KDTree	1.20 $\times$	—	—
LinearListReduction	1.31 $\times$	—	—
List	0.88 $\times$	—	—
MonoTree	1.09 $\times$	—	—
ObjectGraph	0.89 $\times$	—	—
Oct:sumMass	0.65 $\times$	—	—
Oct:sumEnergy	0.67 $\times$	—	—
Oct:countActive	0.67 $\times$	—	—
Oct:countParticles	0.79 $\times$	—	—
Oct:barnesHutPotential	0.61 $\times$	—	—
Oct:fmmPotential	0.76 $\times$	—	—
Oct:scaleEnergy	0.70 $\times$	—	—
Oct:clearFlags	0.75 $\times$	—	—
PiecewiseFunctions	1.00 $\times$	—	—
TernaryTree	0.95 $\times$	—	—
Trie	0.95 $\times$	—	—
ColorOctree	0.84 $\times$	—	—

Table 4: Per-pass performance for **OctTree**, OctTree SoA uses 9 buffers; ColorOctree SoA uses 18 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
barnesHutPotential	F	11/16	31%	0.0427 $\pm$ 0.00006	0.0392 $\pm$ 0.00002	1.09 $\times$
countActive	F	10/16	38%	0.0376 $\pm$ 0.00004	0.0291$\pm$0.00001	1.29$\times$
countParticles	F	8/16	50%	0.0373 $\pm$ 0.00731	0.0266$\pm$0.00008	1.40$\times$
fmmPotential	F	12/16	25%	0.0977 $\pm$ 0.00004	0.0956 $\pm$ 0.00003	1.02 $\times$
sumEnergy	F	12/16	25%	0.0372 $\pm$ 0.00002	0.0333$\pm$0.00010	1.12$\times$
sumMass	F	10/16	38%	0.0351 $\pm$ 0.00004	0.0271$\pm$0.00004	1.29$\times$
clearFlags	M	15/16	6%	0.1539$\pm$0.00021	0.1736 $\pm$ 0.00015	0.89 $\times$
scaleEnergy	M	16/16	0%	0.1499$\pm$0.00028	0.1747 $\pm$ 0.01118	0.86 $\times$
build	?	–	–	0.3425$\pm$0.01864	0.8390 $\pm$ 0.03814	0.41 $\times$
ColorOctree.build	?	–	–	0.3788$\pm$0.02517	0.7001 $\pm$ 0.02336	0.54 $\times$
OctTree_clearFlags.build	?	–	–	0.3525$\pm$0.01524	0.8393 $\pm$ 0.02144	0.42 $\times$
OctTree_countActive.build	?	–	–	0.3428$\pm$0.01271	0.8427 $\pm$ 0.01868	0.41 $\times$
OctTree_countParticles.build	?	–	–	0.3433$\pm$0.01746	0.8378 $\pm$ 0.02868	0.41 $\times$
OctTree_fmmPotential.build	?	–	–	0.3425$\pm$0.02528	0.8373 $\pm$ 0.01643	0.41 $\times$
OctTree_scaleEnergy.build	?	–	–	0.3531$\pm$0.01726	0.8338 $\pm$ 0.01811	0.42 $\times$
OctTree_sumEnergy.build	?	–	–	0.3425$\pm$0.01141	0.8379 $\pm$ 0.02745	0.41 $\times$
OctTree_sumMass.build	?	–	–	0.3436$\pm$0.01490	0.8391 $\pm$ 0.01603	0.41 $\times$
paletteEntriesQuantized	F	13/25	48%	0.0443 $\pm$ 0.00004	0.0335$\pm$0.00007	1.32$\times$
quantizationErrorProxy	F	10/25	60%	0.0476 $\pm$ 0.00003	0.0431$\pm$0.00002	1.10$\times$
Total				3.825	8.083	0.47 $\times$
Geomean						0.71 $\times$

Table 5: Per-pass performance for **Compiler**, ADT has 9 fields, SoA uses 8 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
blockCountPass	F	2/9	78%	0.0123 $\pm$ 0.00002	0.0019$\pm$0.00000	6.39$\times$
branchStatsPass	F	2/9	78%	0.0119 $\pm$ 0.00002	0.0033$\pm$0.00001	3.55$\times$
castInstCountPass	F	3/9	67%	0.0184 $\pm$ 0.00002	0.0013$\pm$0.00001	14.21$\times$
has cycle	F	4/9	56%	0.0247 $\pm$ 0.00094	0.0176$\pm$0.00133	1.40$\times$
instCountPass	F	2/9	78%	0.0123 $\pm$ 0.00006	0.0019$\pm$0.00001	6.39$\times$
latencyModelPass	F	3/9	67%	0.0121 $\pm$ 0.00002	0.0029$\pm$0.00001	4.21$\times$
memoryOpStatsPass	F	3/9	67%	0.0120 $\pm$ 0.00002	0.0037$\pm$0.00001	3.19$\times$
throughputModelPass	F	3/9	67%	0.0122 $\pm$ 0.00002	0.0029$\pm$0.00001	4.16$\times$
verifyIR	F	9/9	0%	0.0126 $\pm$ 0.00002	0.0034$\pm$0.00001	3.70$\times$
stripSideEffectsPass	M	7/9	22%	0.0799$\pm$0.00048	0.0897 $\pm$ 0.00016	0.89 $\times$
targetReturnPass	M	9/9	0%	0.0796$\pm$0.00044	0.0905 $\pm$ 0.00944	0.88 $\times$
build	?	–	–	0.2062 $\pm$ 0.01172	0.2082 $\pm$ 0.02484	0.99 $\times$
Total				0.4941	0.4275	1.16$\times$
Geomean						2.97$\times$

Table 6: Per-pass performance for **DBQuery**, ADT has 15 fields, SoA uses 13 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
countJoins	F	3/15	80%	0.0190 $\pm$ 0.00666	0.0109$\pm$0.00003	1.75$\times$
filterSelectivitySkew	F	5/15	67%	0.0188 $\pm$ 0.00002	0.0124$\pm$0.00003	1.52$\times$
hashJoinPressure	F	5/15	67%	0.0204 $\pm$ 0.00001	0.0129$\pm$0.00005	1.58$\times$
sumCost	F	6/15	60%	0.0188 $\pm$ 0.00003	0.0149$\pm$0.00005	1.27$\times$
sumMemory	F	6/15	60%	0.0189 $\pm$ 0.00002	0.0145$\pm$0.00009	1.30$\times$
sumRows	F	6/15	60%	0.0250 $\pm$ 0.00017	0.0216$\pm$0.00010	1.16$\times$
clearQueryFlags	M	14/15	7%	0.0695$\pm$0.00027	0.0827 $\pm$ 0.00007	0.84 $\times$
scaleCosts	M	15/15	0%	0.0685$\pm$0.00021	0.0835 $\pm$ 0.00010	0.82 $\times$
build	?	–	–	0.2179 $\pm$ 0.02085	0.2349 $\pm$ 0.01594	0.93 $\times$
Total				0.4769	0.4882	0.98 $\times$
Geomean						1.20$\times$

Table 7: Per-pass performance for **DecisionTree**, ADT has 7 fields, SoA uses 6 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
classify Batch	F	5/7	29%	3.306 $\pm$ 0.0207	3.204 $\pm$ 0.0335	1.03 $\times$
classify Depth	F	4/7	43%	0.0470 $\pm$ 0.00005	0.0366$\pm$0.00019	1.28$\times$
classify tree	F	5/7	29%	0.0469 $\pm$ 0.00897	0.0368$\pm$0.00016	1.27$\times$
countFeatureUses	F	3/7	57%	0.0505 $\pm$ 0.00003	0.0319$\pm$0.00005	1.58$\times$
countLeaves	F	2/7	71%	0.0475 $\pm$ 0.00741	0.0249$\pm$0.00863	1.91$\times$
countNodes	F	2/7	71%	0.0476 $\pm$ 0.00002	0.0264$\pm$0.00006	1.81$\times$
countSmallLeaves	F	3/7	57%	0.0510 $\pm$ 0.00004	0.0318$\pm$0.00008	1.61$\times$
inferenceCost	F	2/7	71%	0.0507 $\pm$ 0.00004	0.0290$\pm$0.00002	1.75$\times$
sumImpurity	F	3/7	57%	0.0505 $\pm$ 0.00003	0.0320$\pm$0.00005	1.58$\times$
sumPathLengths	F	3/7	57%	0.0527 $\pm$ 0.00002	0.0335$\pm$0.00008	1.57$\times$
sumSamples	F	3/7	57%	0.0483 $\pm$ 0.00010	0.0319$\pm$0.00005	1.51$\times$
treeDepth	F	2/7	71%	0.0511 $\pm$ 0.00002	0.0283$\pm$0.00005	1.81$\times$
build	?	–	–	0.4205 $\pm$ 0.01295	0.4392 $\pm$ 0.01342	0.96 $\times$
Total				4.270	3.987	1.07 $\times$
Geomean						1.48$\times$

Table 14: Per-pass performance for `PiecewiseFunctions`, ADT has 8 fields, SoA uses 7 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across `-iterate` runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
autorefineMaxLevel	F	4/8	50%	0.0209 $\pm$ 0.00001	0.0195 $\pm$ 0.00005	1.07 $\times$
compressMass	F	3/8	62%	0.0190 $\pm$ 0.00001	0.0117$\pm$0.00544	1.62$\times$
lbDeuxLoadProxy	F	5/8	38%	0.0208 $\pm$ 0.00004	0.0180$\pm$0.00735	1.16$\times$
norm2Estimate	F	5/8	38%	0.0242 $\pm$ 0.00003	0.0178$\pm$0.00001	1.36$\times$
pmapCutHistogram	F	4/8	50%	0.0317 $\pm$ 0.00003	0.0126$\pm$0.00001	2.51$\times$
truncateTolViolations	F	3/8	62%	0.0192 $\pm$ 0.00001	0.0115$\pm$0.00091	1.67$\times$
addConstPW	M	8/8	0%	0.1170 $\pm$ 0.00054	0.1286 $\pm$ 0.00035	0.91 $\times$
diffPW	M	8/8	0%	0.1194 $\pm$ 0.00042	0.1280 $\pm$ 0.00035	0.93 $\times$
build	?	–	–	0.2490 $\pm$ 0.02076	0.2596 $\pm$ 0.01534	0.96 $\times$
Total				0.6211	0.6073	1.02 $\times$
Geomean						1.28$\times$

Table 15: Per-pass performance for `TernaryTree`, ADT has 5 fields, SoA uses 3 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across `-iterate` runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
sum tree	F	5/5	0%	0.0304 $\pm$ 0.00011	0.0289 $\pm$ 0.00011	1.05 $\times$
add 1 tree	M	5/5	0%	0.0840 $\pm$ 0.00013	0.0775 $\pm$ 0.00020	1.08 $\times$
build	?	–	–	0.2126 $\pm$ 0.00094	0.2219 $\pm$ 0.02075	0.96 $\times$
Total				0.3270	0.3283	1.00 $\times$
Geomean						1.03 $\times$

Table 16: Per-pass performance for `Trie`, ADT has 10 fields, SoA uses 9 buffers. Times are median per iteration (s); $\pm$ shows standard error of the mean across `-iterate` runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup $>1\times$ means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	AoS med $\pm$ err	SoA med $\pm$ err	Speedup
autocompleteTopKProxy	F	5/10	50%	0.0145 $\pm$ 0.00002	0.0079$\pm$0.00014	1.82$\times$
countLazyNodes	F	4/10	60%	0.0130 $\pm$ 0.00002	0.0067$\pm$0.00001	1.95$\times$
countTerminals	F	3/10	70%	0.0122 $\pm$ 0.00002	0.0058$\pm$0.00004	2.09$\times$
sumPrefixFreq	F	3/10	70%	0.0124 $\pm$ 0.00003	0.0058$\pm$0.00002	2.16$\times$
sumSubtreeHints	F	3/10	70%	0.0123 $\pm$ 0.00002	0.0057$\pm$0.00001	2.15$\times$
decayTrieStats	M	10/10	0%	0.0745$\pm$0.00035	0.0850 $\pm$ 0.00949	0.88 $\times$
resetTraversalState	M	10/10	0%	0.0751 $\pm$ 0.00038	0.0827 $\pm$ 0.00023	0.91 $\times$
build	?	–	–	0.2074 $\pm$ 0.01512	0.2111 $\pm$ 0.02128	0.98 $\times$
Total				0.4213	0.4108	1.03 $\times$
Geomean						1.51$\times$